Prio: Private, Robust, and Scalable Computation of Aggregate Statistics

2026/3/6

GOTO Naoto

Outline

Preleminaries 3
Prio 23
SNIP 28
Affine-Aggregatable Encodings (AFEs) 39
Discussion 47

Preleminaries

Secure Computation

Secure compuation is a technique to run a computation while keeping th input data private.

Mainly, three techniques are known for secure computation:

- homomorphic encryption
- trusted execution environment (TEE)
- multi-party computation

	Transmission / Storage	Computation
Encryption (RSA, AES)	○	✕
Secure Computation	○	○

Table 1: Operations Possible While Data Remains Encrypted

Secure Computation (ii)

1. Homomorphic Encryption

Definition 1: Homomorphic Encryption.

An encryption scheme $\mathcal{E}$ is said to have the homomorphic property if, for an operation $\circ$ on the plaintext space $\mathcal{M}$ and an operation $\star$ on the ciphertext space $\mathcal{C}$, the following property holds:

$$\boxed{\forall m_1, m_2 \in \mathcal{M}: \quad \mathrm{Enc}(m_1) \star \mathrm{Enc}(m_2) = \mathrm{Enc}(m_1 \circ m_2)}$$

That is, decrypting the result of an operation on ciphertexts yields the same result as applying the corresponding operation to the plaintexts.

RSA is an example of a homomorphic encryption scheme where a computation is multiplication.

$$\mathrm{Enc}(m) = m^e \bmod n$$
$$\mathrm{Enc}(m_1)\,\mathrm{Enc}(m_2) = \mathrm{Enc}(m_1 m_2)$$

Secure Computation (iii)

2. Trusted Execution Environment (TEE)

Trusted Execution Environment (TEE) is a **hardware support** that provides an isolated environment for executing code and processing data. TEE is employed in various applications, such as managing secure keys, mobile payments, and biometric authentication.

ex) Intel SGX, ARM TrustZone

SORRY for a little content here…..

Secure Computation (iv)

3. Multi-Party Computation

Secure Multi-Party Computation (MPC) allows multiple parties to **jointly compute** a function over their inputs while keeping those inputs private.

Let there be n parties $P_1, ..., P_n$, where party P_i holds a private input x_i. Given a public function f, the goal of MPC is to compute

$$y = f(x_1, x_2, ..., x_n)$$

such that, apart from what can be inferred from the output y (and any allowed auxiliary information), no party learns anything about the other parties' inputs.

Spliting inputs into shares and distributing them among parties is a common approach in MPC.

Secure Computation (v)

Data a is splited into three shares, and every administrator computes its share.

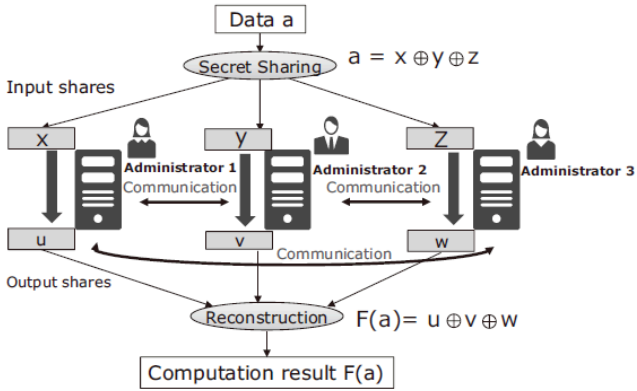


Figure 1: Secure Multi-Party Computation (MPC)[1]

[1] https://www.nec.com/en/global/techrep/journal/g16/n02/images/160211_01.png

Multi-Party Computation

The flow of MPC is as follows:

1. Each party splits its private input into shares and distributes them to other parties.
2. Each party performs computations on the shares it has received. These computations may include additions, multiplications, or more complex operations depending on the protocol.
3. After the computations, the parties exchange the results of their computations.
4. Finally, the parties combine the results to obtain the final output.

Here are two main techniques for splitting data into shares:

- Shamir's Secret Sharing
- Additive Secret Sharing

Shamir's Secret Sharing

Shamir's Secret Sharing is a (t, n) threshold scheme that splits a secret into n shares such that any t shares can reconstruct the secret, while fewer than t shares reveal no information about it. The scheme is widely used in cryptography to distribute trust and tolerate failures or compromises.

The idea is to encode the secret as the constant term of a random polynomial over a finite field. Choose a prime p (or, more generally, a finite field $\mathbb{F}$) and represent the secret as $s \in \mathbb{F}$. Then sample a random polynomial

$$f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1}$$

where the coefficients $a_1, \cdots, a_{t-1}$ are chosen uniformly at random from $\mathbb{F}$. For n distinct nonzero points $x_1, \cdots, x_n$, the i-th share is the pair $(x_i, f(x_i))$.

To reconstruct the secret, any set of at least t parties combines their shares and uses Lagrange interpolation to recover $f(0)$, which equals the secret s.

Security follows from the fact that **with fewer than t points, the polynomial is underdetermined**: infinitely many degree-$(t-1)$ polynomials match those points.

Shamir's Secret Sharing (ii)

Example of Shamir's Secret Sharing

We build a $(t, n) = (3, 5)$ Shamir secret sharing scheme over the finite field $\mathbb{F}_{17}$. Let the secret be $s = 5$ and choose a random degree-2 polynomial

$$f(x) = s + a_1 x + a_2 x^2 \pmod{17}$$

with $a_1 = 2$ and $a_2 = 7$. So,

$$f(x) = 5 + 2x + 7x^2 \pmod{17}$$

Step 1: Share generation

We pick distinct nonzero x values and compute $y_i = f(x_i)$.

$(x_i, f(x_i))$	value in $\mathbb{F}_{17}$
$(1, f(1))$	$f(1) = 5 + 2 \cdot 1 + 7 \cdot 1^2 = 14$
$(2, f(2))$	$f(2) = 5 + 2 \cdot 2 + 7 \cdot 2^2 = 37 \equiv 3$
$(3, f(3))$	$f(3) = 5 + 2 \cdot 3 + 7 \cdot 3^2 = 74 \equiv 6$
$(4, f(4))$	$f(4) = 5 + 2 \cdot 4 + 7 \cdot 4^2 = 125 \equiv 6$
$(5, f(5))$	$f(5) = 5 + 2 \cdot 5 + 7 \cdot 5^2 = 190 \equiv 3$

Step 2: Reconstruction from 3 shares

Take any 3 shares, e.g. $(1, 14), (2, 3), (3, 6)$.

Using Lagrange interpolation over the field, we recover $f(0)$:

Shamir's Secret Sharing (iii)

$$f(0) = \sum_{i=1}^{3} y_i \ell_{i(0)} \pmod{17}, \quad \ell_{i(x)} = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

Compute the basis values at $x = 0$:

- For $x_1 = 1$:

$$\ell_1(0) = \frac{0-2}{1-2} \cdot \frac{0-3}{1-3} = \frac{-2}{-1} \cdot \frac{-3}{-2} = \frac{6}{2}.$$

In $\mathbb{F}_{17}$, $2^{-1} \equiv 9$ (since $2 \cdot 9 = 18 \equiv 1$), so

$$\ell_1(0) \equiv 6 \cdot 9 = 54 \equiv 3.$$

- For $x_2 = 2$:

$$\ell_2(0) = \frac{0-1}{2-1} \cdot \frac{0-3}{2-3} = \frac{-1}{1} \cdot \frac{-3}{-1} = \frac{3}{-1}.$$

Since $(-1) \equiv 16$ and $16^{-1} \equiv 16$ in $\mathbb{F}_{17}$,

$$\ell_2(0) \equiv 3 \cdot 16 = 48 \equiv 14.$$

- For $x_3 = 3$:

$$\ell_3(0) = \frac{0-1}{3-1} \cdot \frac{0-2}{3-2} = \frac{-1}{2} \cdot \frac{-2}{1} = \frac{2}{2}.$$

Thus

$$\ell_3(0) \equiv 1.$$

So,

$$f(0) \equiv 14 \cdot 3 + 3 \cdot 14 + 6 \cdot 1$$
$$= 42 + 42 + 6 = 90 \equiv 5 \pmod{17}$$

Therefore the reconstructed secret is $s = f(0) = 5$.

Shamir's Secret Sharing (iv)

ADD computation

Assume two secrets $s_1, s_2 \in \mathbb{F}_p$ are shared by polynomials $f_1(x)$ and $f_2(x)$ of degree at most $(t-1)$.

Each participant i holds the shares

$$y_{\{1,i\}} = f_1(i), \quad y_{\{2,i\}} = f_2(i).$$

Define the polynomial

$$f_+(x) = f_1(x) + f_2(x).$$

Since polynomial addition is coefficient-wise, $f_+(x)$ also has degree at most $(t-1)$ and

$$f_+(0) = f_1(0) + f_2(0) = s_1 + s_2.$$

Moreover, for each participant i,

$$y_{\{+,i\}} = y_{\{1,i\}} + y_{\{2,i\}} = f_1(i) + f_2(i) = f_+(i).$$

Thus, locally adding their shares is sufficient for the participants to obtain shares of the sum $s_1 + s_2$ without revealing their individual secrets.

Shamir's Secret Sharing (v)

MULTIPLICATION computation

Define the polynomial

$$f_*(x) = f_1(x) f_2(x).$$

Since the product of two polynomials of degree at most $(t-1)$ has degree at most $2(t-1)$, the polynomial $f_*(x)$ satisfies

$$f_*(0) = f_1(0) f_2(0) = s_1 s_2.$$

Moreover, for each participant i,

$$y_{\{*,i\}} = y_{\{1,i\}} y_{\{2,i\}} = f_1(i) f_2(i) = (f_1 f_2)(i) = f_*(i).$$

Thus, by locally multiplying their shares, participants obtain shares of the secret $s_1 s_2$. The resulting polynomial has degree at most $2(t-1)$.

To restore the threshold t, a degree reduction protocol is applied, which involves sharing the shares of $f_*(i)$ using a new random degree-$(t-1)$ polynomial and reconstructing $f_*(0)$ from the new shares.

Additive Secret Sharing

Additive Secret Sharing is a simple secret sharing scheme in which a secret is split into multiple shares such that the **sum of the shares equals the secret**. It is commonly used in secure multi-party computation (MPC) due to its simplicity and efficient support for linear operations.

The scheme typically works over a finite field $\mathbb{F}$ (for example Z_p for a prime p). Let the secret be represented as $s \in \mathbb{F}$.

The goal is to generate n shares $x_1, x_2, \cdots, x_n \in \mathbb{F}$ such that

$$x_1 + x_2 + \cdots + x_n = s$$

To generate the shares, sample $n - 1$ values uniformly at random from $\mathbb{F}$:

$$x_1, x_2, \cdots, x_{n-1} \leftarrow \mathbb{F}$$

Then compute the final share as

$$x_n = s - (x_1 + x_2 + \cdots + x_{n-1})$$

Each party P_i receives one share x_i.

Additive Secret Sharing (ii)

MULTIPLICATION computation (Beaver Triples)

All computations below are performed over the finite field $\mathbb{F}$.

A Beaver triple consists of three random values (a, b, c) such that $c = ab$.

These values are secret-shared among the participants:

$$a = a_1 + a_2 + \cdots + a_n$$

$$b = b_1 + b_2 + \cdots + b_n$$

$$c = c_1 + c_2 + \cdots + c_n$$

Each participant i holds the shares (a_i, b_i, c_i). The triple is generated during a preprocessing phase and is independent of the secrets that will later be multiplied.

Goal Suppose two secrets $x = x_1 + x_2 + \cdots + x_n$ and $y = y_1 + y_2 + \cdots + y_n$ are shared among the participants. The goal is to obtain shares of the product xy without revealing x or y.

Additive Secret Sharing (iii)

Step 1: Mask the Secrets

Each participant locally computes

$$d_i = x_i - a_i$$

$$e_i = y_i - b_i$$

Then the parties reconstruct the public values

$$d = \sum_i d_i = x - a$$

$$e = \sum_i e_i = y - b$$

Since a and b are random and unknown to the parties, revealing d and e does not leak information about x or y.

Step 2: Compute Shares of the Product

Using the identity

$$xy = (a + d)(b + e)$$

we expand

$$xy = ab + db + ea + de$$

Because $c = ab$, we can rewrite this as

$$xy = c + db + ea + de$$

Each participant then computes a share of the product as

$$z_i = c_i + db_i + ea_i$$

Additive Secret Sharing (iv)

and one designated participant (or all parties with a correction term) adds the public value de so that

$$z_1 + z_2 + \cdots + z_n = xy$$

Thus the shares

$$z_1, z_2, \cdots, z_n$$

represent the secret

$$xy$$

without revealing any information about x or y to the parties.

Where to use MPC?

Privacy-preserving Medical Research

Hospitals can jointly analyze patient datasets (e.g., cancer or COVID-19 statistics) using MPC so that sensitive patient records remain inside each institution. MPC enables researchers to compute aggregate statistics or train machine learning models on distributed data without moving hospitals' data outside their institutions.

Example: Kidney Exchange Program[2]

Privacy-preserving Auctions

Organizations can conduct auctions where participants' bids remain confidential while still determining the correct market outcome. Using MPC, bidders submit encrypted bids, and the system computes the auction result (such as the winning bidder or market-clearing price) without revealing individual bids. This removes the need for a trusted auctioneer and ensures fairness while protecting sensitive business information.

Example: Danish Sugar Beet Auction[3]

[2] https://link.springer.com/article/10.1186/s12911-022-01994-4
[3] https://www.partisia.com/blog/worlds-first-commercial-use-of-multi-party-computation-mpc

MPC for Privacy-Preserving Measurement

Modern Internet services often require large-scale measurements in order to improve systems, train machine learning models, or understand software reliability. However, collecting raw user data can create serious privacy risks. Many measurements—such as browser crash statistics, keyboard prediction training data, or health exposure notifications—require aggregated statistics rather than individual user data.

To address this challenge, the Internet Engineering Task Force (IETF) established the **Privacy Preserving Measurement (PPM) Working Group**. The goal of this working group is to design standardized protocols that enable useful data collection while minimizing the exposure of information about individual users.

The central protocol developed in this effort is the **Distributed Aggregation Protocol (DAP)**.

Privacy Preserving Measurement (PPM) Working Group

PPM is a framework that allows organizations to compute **aggregate statistics** from sensitive user data **without revealing individual measurements**.

Motivation

- Many measurements involve **sensitive data**
- Example:
  - Browser vendors measuring rendering errors
  - Public health agencies measuring disease exposure
- Traditional approach:
  - Collect individual data in plaintext
  - Aggregate afterward

→ **Risk of privacy leakage**

Key Idea

- Clients send encrypted / secret-shared measurements
- One or more **non-colluding servers** process the data
- Only **aggregate statistics** are revealed

Examples of supported statistics:

- Average values
- Frequency / counts of events

Privacy Preserving Measurement (PPM) Working Group (ii)

PPM Protocol Goals

- Secure client submission of measurements
- Verification of measurement validity
- Privacy-preserving computation of aggregates
- Protection against abuse
  - leakage of individual measurements
  - denial-of-service attacks

PPM protocols rely on cryptographic primitives defined by the CFRG and are being standardized by the IETF PPM Working Group.

The **Distributed Aggregation Protocol (DAP)** is the primary protocol developed by the PPM working group to enable privacy-preserving data aggregation. It applies techniques from **multi-party computation (MPC)** by splitting each client's measurement into cryptographic shares processed by multiple non-colluding aggregators, ensuring that only the final aggregate result is revealed while individual measurements remain hidden.

Prio

Paper Overview

Title & Authors: "Prio: Private, Robust, and Scalable Computation of Aggregate Statistics", Henry Corrigan-Gibbs and Dan Boneh

Position: **A milestone paper in DAP's history**

	Privacy	Scalability	Robustness
MPC traditional protocol (SPDZ,BGW)	○	computation is heavy	○
Differential Privacy (RAPPOR)	noisy	○	✕[4]
Prio	○	○	○

Table 2: Comparison of Prio with Traditional MPC and Differential Privacy

Definition 2: Robustness.

A protocol is robust if malicious clients cannot significantly bias the aggregated result beyond their proportional contribution.

[4] PKI is a solution for verifying clients' identities, not clients' inputs. Thus, combining PKI with a non-robust protocol such as RAPPOR does not achieve robustness.

Whye robustness is needed?

In privacy-preserving measurement systems, servers aggregate client data without seeing individual values. Because of this, malicious clients may try to send **invalid or malformed inputs** that could distort the aggregated result.

For example, in a system that collects car speeds, a malicious client might submit an impossible value such as **100,000 km/h**, even though the valid range is between **0 and 200 km/h**. If such inputs are accepted, they can significantly bias the statistics.

Robustness is therefore needed to ensure that servers can detect and reject syntactically invalid submissions while preserving privacy. This prevents attackers from corrupting the aggregation with out-of-range or malformed data.

However, robustness does **not guarantee truthfulness**. A malicious client may still submit an incorrect but valid value (e.g., reporting 100 km/h instead of the actual 80 km/h).

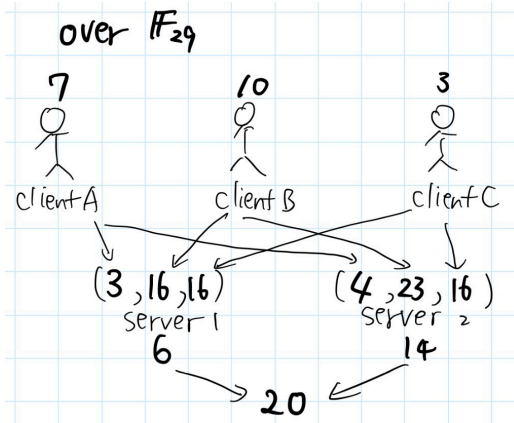
Contributions

- introduce secret-shared non-interactive proofs (SNIPs), a new type of information-theoretic zero-knowledge proof, optimized for the client/server setting, (**Robustness**)
- present affine-aggregatable encodings, a frameworkthat unifies many data-encoding techniques used in prior work on private aggregation, and
- demonstrate how to combine these encodings with SNIPs to provide robustness and privacy in a largescale data-collection system. (**Scalability**)

It can be said that Prio's main contribution is the design of SNIPs, which balances the trade-off between robustness and scalability in the context of privacy-preserving measurement.

Simple Architecture

Simple additive secret sharing-based protocol for computing the sum of client inputs.



However, this simple architecture is not robust. –> **SNIP**

And, we need more complex statistics than just sums. –> **Affine-aggregatable encodings**

SNIP

Arithmetic Circuits

An arithmetic circuit C over a finite field $\mathbb{F}$ takes as input a vector

$$x = \langle x^{(1)}, \ldots, x^{(L)} \rangle \in \mathbb{F}^L$$

and produces a single field element as output.

The circuit is represented as a directed acyclic graph (DAG). Each vertex in the graph is one of the following:

- Input vertex
- Gate vertex
- Output vertex

Input Vertices

Input vertices have in-degree 0 and are labeled with either:

- a variable in $\{x^{(1)}, \ldots, x^{(L)}\}$, or
- a constant in $\mathbb{F}$.

Arithmetic Circuits (ii)

Gate Vertices

Gate vertices have in-degree 2 and are labeled with one of the operations:

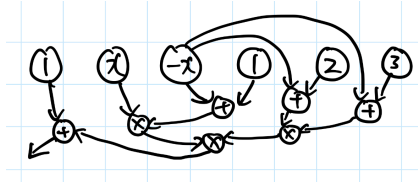
- $+$ (addition)
- $\times$ (multiplication)

Output Vertex: The circuit has a single output vertex, which has out-degree 0.

Example This circuit computes the function:

$$C(x) = 1 - (x - 0)(x - 1)(x - 2)(x - 3)$$

It verfies that the input x is in the set $\{0, 1, 2, 3\}$.



Setting and Goal

Setting

We want to verify a client input on an **arithmetic circuit**.

Multiplication gates of the circuit are ordered topologically: $t = 1, 2, \ldots, M$

For each gate we denote:

- left input: u_t
- right input: v_t
- output: w_t

Correct computation satisfies: $w_t = u_t v_t$

The client input x is secret-shared among the servers.

Servers hold: $[x]_1, [x]_2, \ldots, [x]_s$

Goal

Verify that $\mathrm{Valid}(x) = 1$ **without revealing x.**

Step 1: Polynomial Encoding of the Circuit

The client evaluates the arithmetic circuit locally. Using interpolation the client constructs polynomials

$$f(t) = u_t$$
$$g(t) = v_t$$

and

$$h(t) = f(t)g(t)$$

$f(0), g(0)$ are determined randomly over $\mathbb{F}$ by the client.

f, g are random polynomials of degree at most M that pass through the points (t, u_t) and (t, v_t) for $t = 1, ..., M$.

h is a random polynomial of degree at most $2M$ that passes through the points (t, w_t) for $t = 1, ..., M$.

Step 1: Polynomial Encoding of the Circuit (ii)

What the Client Sends

The client **secret-shares** the following values to the servers:

- share of x
- shares of $f(0)$
- shares of $g(0)$
- shares of the coefficients of $h(t)$
- shares of a Beaver triple (a, b, c) with $ab = c$

Each server i receives

$$[x]_i, \quad [f(0)]_i, \quad [g(0)]_i, \quad [h]_i, \quad [a]_i, \quad [b]_i, \quad [c]_i$$

No single server learns the actual values.

Step 2: Reconstruction by the Servers

Using

- the circuit structure
- the shares of the coefficients of $h(t)$
- the shares of $f(0)$ and $g(0)$
- the share of x

Tracing the circuit with these shares, each server locally reconstructs shares of the polynomials

$$\left[\hat{f}\right]_i, \quad [\hat{g}]_i$$

The servers now collectively hold shares of

$$\hat{f}, \quad \hat{g}, \quad \hat{h}$$

These represent the client's claimed computation.

Step 3: Checking the Polynomial Relation

Servers must verify

$$\hat{f}(t)\hat{g}(t) = \hat{h}(t)$$

for all t.

Instead of checking all points, they perform a randomized identity test.

A server samples $r \in \mathbb{F}$ and broadcasts r to the others.

Each server locally computes

$$\left[\hat{f}(r)\right]_i, \quad [\hat{g}(r)]_i, \quad \left[\hat{h}(r)\right]_i$$

Secure Multiplication

The values $\left[\hat{f}(r)\right]$ and $[\hat{g}(r)]$ are secret-shared.

Servers compute the product using the Beaver triple (a, b, c) with $ab = c$.

Step 3: Checking the Polynomial Relation (ii)

Each server broadcasts small masked values (as in Beaver's protocol).

From this interaction the servers obtain shares of

$$\left[\hat{f}(r)\hat{g}(r)\right]$$

Finally they open

$$\sigma = \hat{f}(r)\hat{g}(r) - \hat{h}(r)$$

If $\sigma = 0$, the check passes.

If a Client Cheats

Suppose the client falsifies a gate output $\hat{w}_t \neq u_t v_t$

Define the **first incorrect gate**

$$t_0 = \min\{t \texttt{ mid } \hat{w}_t \neq u_t v_t\}$$

For all earlier gates the computation is correct.

Therefore the reconstructed polynomials satisfy

$$\hat{f}(t_0) = u_{t_0}$$
$$\hat{g}(t_0) = v_{t_0}$$

But the client submitted $\hat{h}(t_0) = \hat{w}_{t_0}$ so

$$\hat{f}(t_0)\hat{g}(t_0) \neq \hat{h}(t_0)$$

and thus $fg \neq h$

Randomized Detection

Servers evaluate the identity at a random point $r \in \mathbb{F}$

Define the polynomial

$$D(t) = \hat{f}(t)\hat{g}(t) - \hat{h}(t)$$

If the client cheated then

$$D(r) = 0$$

only with probability

$$\leq \frac{2M}{|\mathbb{F}|}$$

by the **Schwartz–Zippel lemma**. Thus cheating is detected with overwhelming probability.

Key Insight

SNIPs reduce **circuit verification** to **a single randomized polynomial identity check.**

Affine-Aggregatable Encodings (AFEs)

Setting and Goal

We already know how to calculate the sum of client inputs with Prio. BUT,....

The fundamental challenge:
Most useful statistics are **not** simple sums. How can we compute complex aggregation functions like:

- Frequency histograms (requires counting occurrences)
- Linear regression (requires sums of products)
- Maximum/minimum values

The AFE Solution

Affine-Aggregatable Encodings (AFEs) provide a framework where:

1. Each client **encodes** its private value x_i into a specially designed vector
2. Servers compute the **sum** of these encoded vectors (compatible with secret sharing)
3. Servers **decode** the summed encoding to recover $f(x_1, ..., x_n)$

By cleverly encoding data, we can **reduce complex aggregation functions to simple summation**.

AFE Definition and Computable Functions

Formal Definition

An AFE is defined with respect to:

- A finite field $\mathbb{F}$
- Integers $k, k' \in \mathbb{N}$ where $k' \leq k$
- Data domain $\mathcal{D}$ and aggregate range $\mathcal{A}$
- Aggregation function $f : \mathcal{D}^n \to \mathcal{A}$

An AFE consists of three algorithms:

Encode: $\mathcal{D} \to \mathbb{F}^k$

- Maps a data value to its encoding

Valid: $\mathbb{F}^k \to \{0, 1\}$

- Returns 1 if the input is a valid encoding of some $x \in \mathcal{D}$
- Can be represented as an arithmetic circuit

Decode: $\mathbb{F}^{k'} \to \mathcal{A}$

- Takes $\sigma = \sum_{i=1}^{n} \mathrm{Trunc}_{k'}(\mathrm{Encode}(x_i))$
- Outputs $f(x_1, ..., x_n)$

Note: Encoding uses all k components for validation, but only k' components are needed for decoding.

Security Properties

Correctness: For all $(x_1, ..., x_n) \in \mathcal{D}^n$:

$$\mathrm{Decode}\left(\sum_{i=1}^{n} \mathrm{Trunc}_{k'}(\mathrm{Encode}(x_i))\right) = f(x_1, ..., x_n)$$

Soundness: $\mathrm{Valid}(e) = 1 \iff \exists x \in \mathcal{D} : e = \mathrm{Encode}(x)$

AFE Definition and Computable Functions (ii)

Catalog of Computable Functions

Prio supports AFEs for:

Basic statistics:

- Integer sum, mean, product, geometric mean
- Variance and standard deviation
- Min and max (exact for small ranges, approximate for large)

Boolean operations:

- OR and AND (with high probability)

Frequency analysis:

- Frequency counts over small domains
- Approximate counts using count-min sketches

Set operations:

- Set intersection and union

Machine learning:

- Linear regression (arbitrary dimensions)
- Least-squares fitting
- Model evaluation (R^2 coefficient)

Combining these AFEs allows Prio to compute a wide range of aggregate statistics.

Example 1: b-bit Integer Summation

Construction

Let each client hold an integer $0 \leq x \leq 2^b - 1$. We work over a finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n \cdot 2^b$ to prevent overflow.

Encoding: For input x, compute binary representation $(\beta_0, \beta_1, ..., \beta_{b-1}) \in \{0,1\}^b$ where $x = \sum_{i=0}^{b-1} 2^i \beta_i$. Output:

$$\text{Encode}(x) = (x, \beta_0, \beta_1, ..., \beta_{b-1}) \in \mathbb{F}^{b+1}$$

Validation: The arithmetic circuit checks two properties:

1. **Bit constraints**: Each β_i is binary: $\beta_i(1 - \beta_i) = 0$ for all i
2. **Consistency**: The bits represent x: $x = \sum_{i=0}^{b-1} 2^i \beta_i$

This requires exactly b multiplication gates (one per bit check).

Decoding: Given $\sigma = \sum_{i=1}^{n} \text{Trunc}_1(\text{Encode}(x_i)) = \sum_{i=1}^{n} x_i$, output σ.

Example 1: b-bit Integer Summation (ii)

Extensions

Mean: Compute sum, then divide by n over the rationals.

Product/Geometric mean: Instead of encoding x directly, encode $\log_2(x)$ using b-bit representation. The sum of logs gives the log of the product.

Variance: Use the identity $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$. Each client encodes (x, x^2) and validates both the bit representations and that the second component is indeed the square of the first. Decode to get both sum and sum-of-squares, then compute variance.

Example 2: Frequency Counts

Problem Statement

Each client holds a value $x \in \mathcal{D} = \{0, 1, ..., B-1\}$ (small discrete domain). The goal is to output a histogram: a length-B vector $\boldsymbol{v}$ where $v[j]$ counts how many clients have value j.

This goes beyond simple summation—we need to **count occurrences** of each possible value in the domain.

Construction

Work over field $\mathbb{F}$ with $|\mathbb{F}| \geq n$.

Encoding: For input $x \in \{0, ..., B-1\}$, create a one-hot vector:

$$\text{Encode}(x) = (\beta_0, \beta_1, ..., \beta_{B-1}) \in \mathbb{F}^B$$

where $\beta_i = 1$ if $x = i$, and $\beta_i = 0$ otherwise.

Validation: The circuit checks:

1. **Bit constraints**: $\beta_i(1 - \beta_i) = 0$ for all i (each component is binary)
2. **One-hot constraint**: $\sum_{i=0}^{B-1} \beta_i = 1$ (exactly one bit is set)

Example 2: Frequency Counts (ii)

This requires B multiplication gates for the bit checks. The sum can be checked with a linear constraint (no multiplications needed).

Decoding: Given $\sigma = \sum_{i=1}^{n} \text{Encode}(x_i)$, simply output σ. The j-th component equals:

$$\sigma[j] = \sum_{i=1}^{n} \beta_{i,j} = |\{i : x_i = j\}|$$

which is exactly the count of clients with value j.

Why One-Hot Constraint Works with Secret Sharing

A crucial question: how can servers verify the one-hot constraint $\sum_{i=0}^{B-1} \beta_i = 1$ when each server only sees shares of the encoding?

–> The one-hot constraint is a **linear equation**, which is preserved under secret sharing

Discussion

My Thoughts

- Prio presented several use cases of MPC. One key point emphasized is that distributing the computation across multiple servers helps **reduce the risk of privacy leakage** caused by hacking or data breaches. While a strength of Prio is the absence of a central registry, similar to the debates around Web3, it is worth considering **how realistic the assumption is that trusting a central registry is inherently dangerous**.
- Differential privacy provides privacy guarantees only in a probabilistic sense. Therefore, if strict correctness of the computed values is required, MPC may be preferable. However, from a computational perspective, differential privacy is significantly more efficient, so the choice between the two should depend on the requirements of the system being built.